



FEATURE PYRAMID NETWORKS

FAVOUR ONYEDIKACHI UMEH

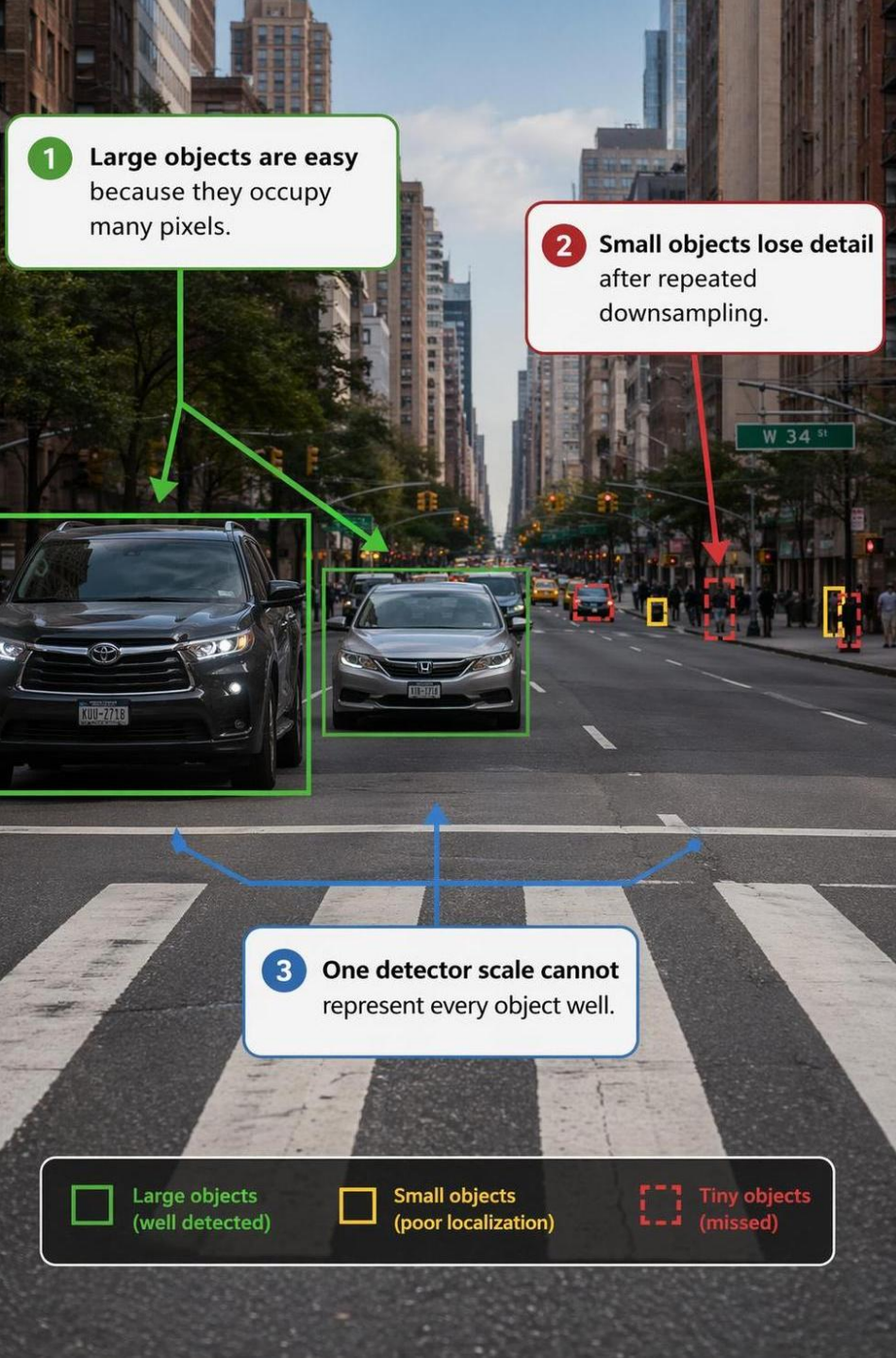
ARI5118 – DEEP LEARNING FOR COMPUTER VISION
DEPARTMENT OF ARTIFICIAL INTELLIGENCE
UNIVERSITY OF MALTA



A visual walkthrough of multi-scale detection



WHY SCALE BREAKS DETECTORS



Large objects are easy because they occupy many pixels.

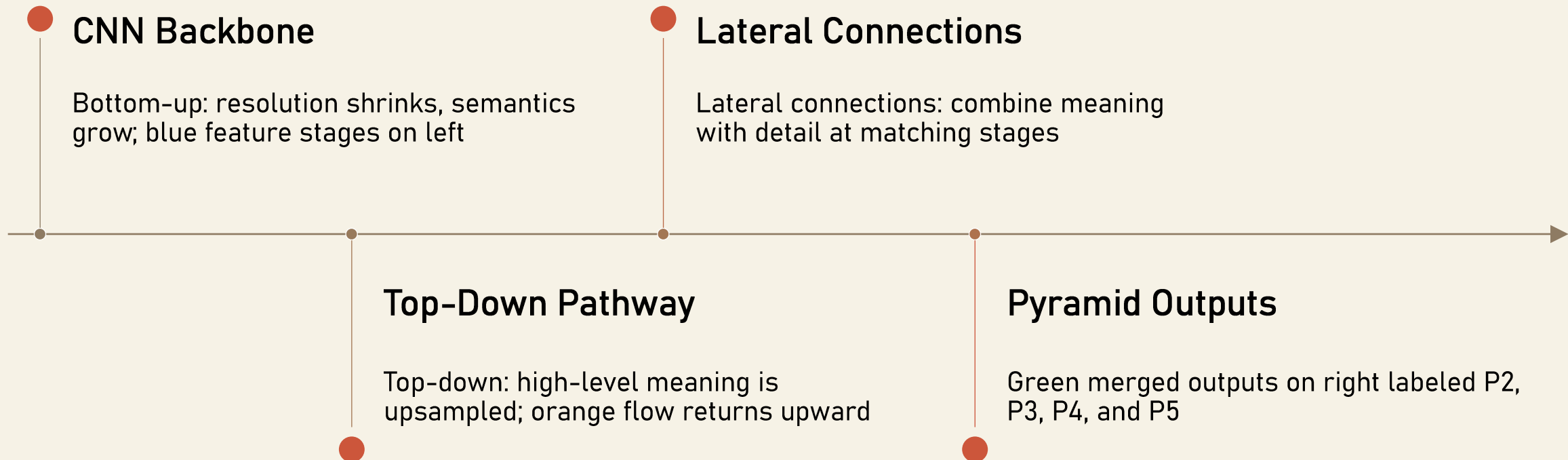
Small objects lose detail after repeated downsampling.

One detector scale cannot represent every object well.

The core problem is not just recognition—it is preserving useful features across scale.

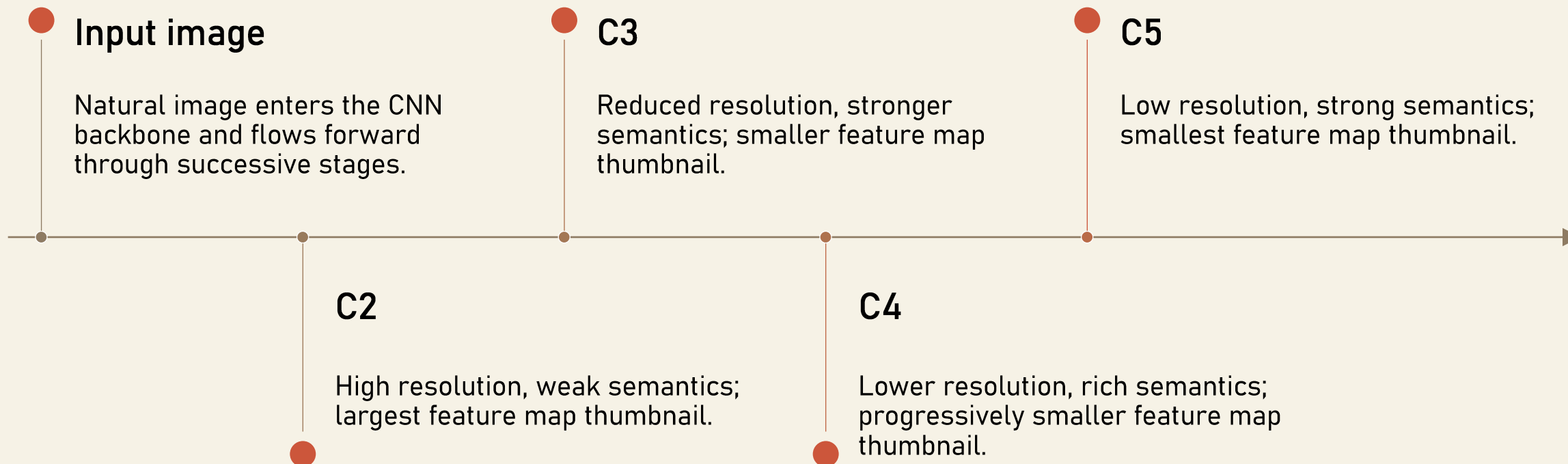


CORE IDEA OF FPN



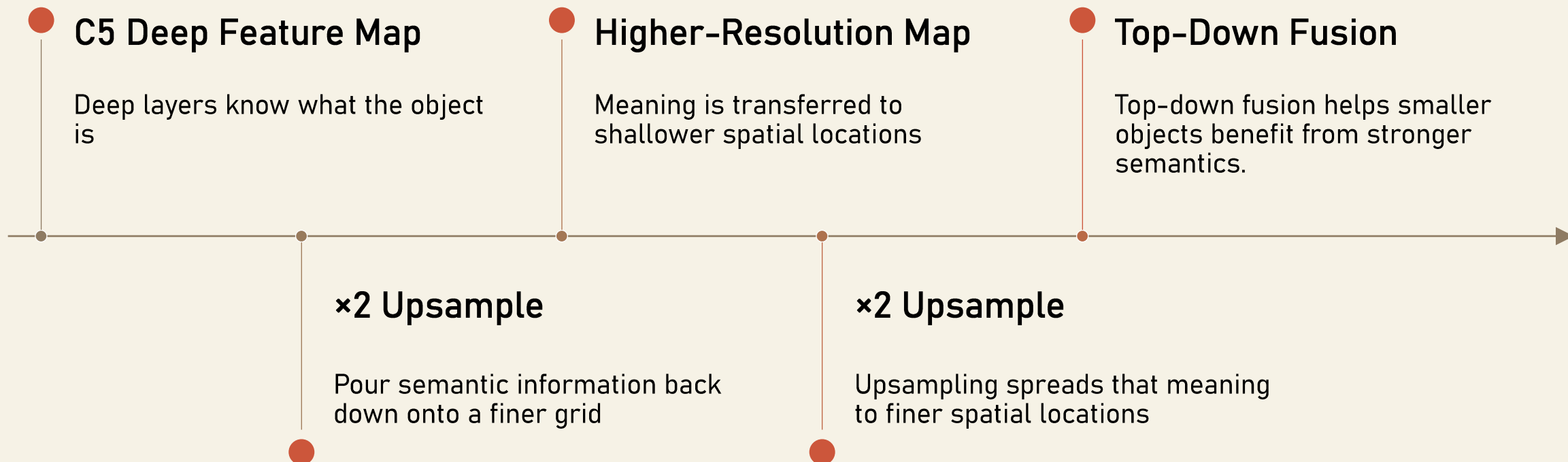


STEP 1: BOTTOM-UP PATHWAY



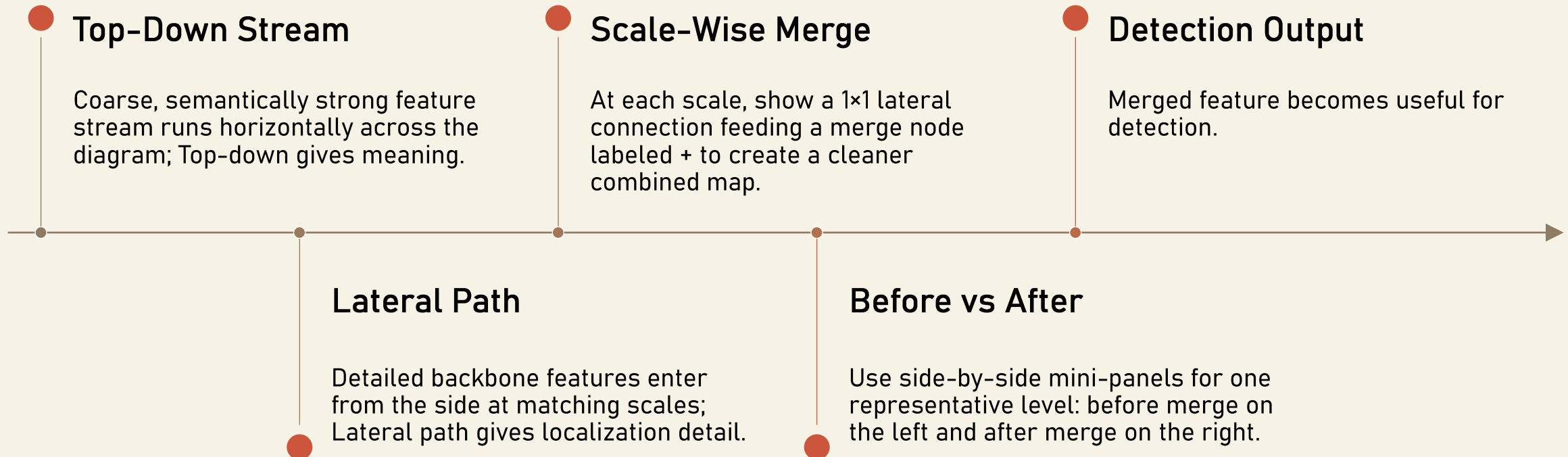


STEP 2: TOP-DOWN PATHWAY





STEP 3: LATERAL CONNECTIONS





THE FINAL PYRAMID: P2 TO P5



P2

Small objects

Highest spatial resolution output feature map

Same strong semantics across levels



P3

Medium-small

High-resolution output feature map

Different spatial resolutions for different object sizes



P4

Medium-large

Lower-resolution output feature map

Detectors can choose the right level



P5

Large objects

Lowest-resolution output feature map

Instead of forcing one feature map to do everything



FPN VS. IMAGE PYRAMIDS

Visual comparison of repeated multi-scale backbone passes versus single-pass FPN reuse, with FPN favored on speed, efficiency, and semantic consistency.

1

IMAGE PYRAMIDS

Resize the same image to multiple scales

2

FPN

Show one input feeding a shared CNN backbone

3

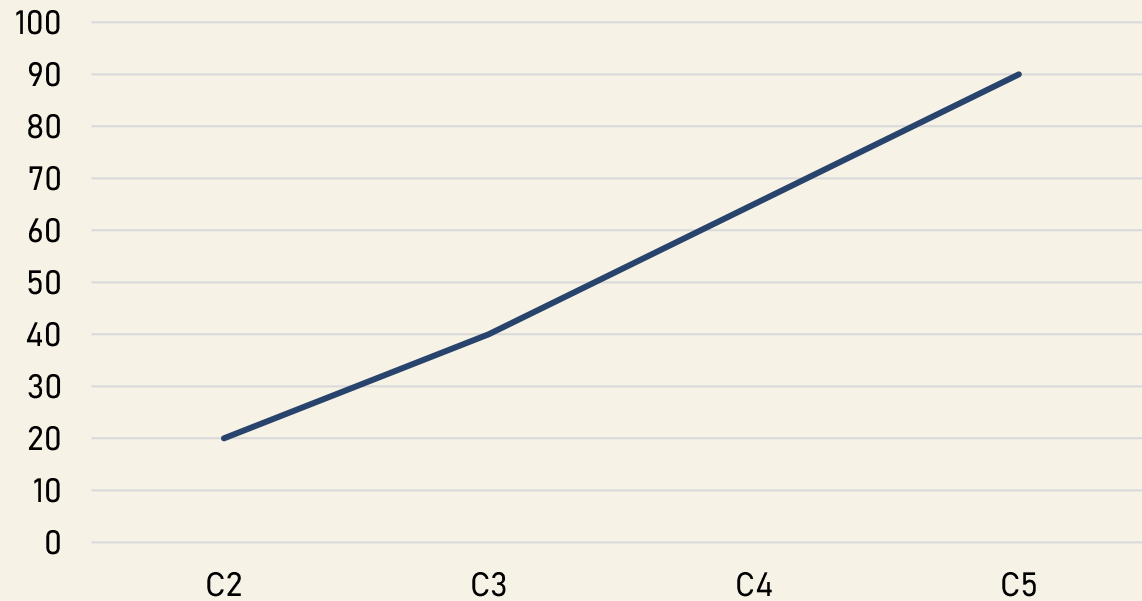
FPN ADVANTAGES

Badge: Speed

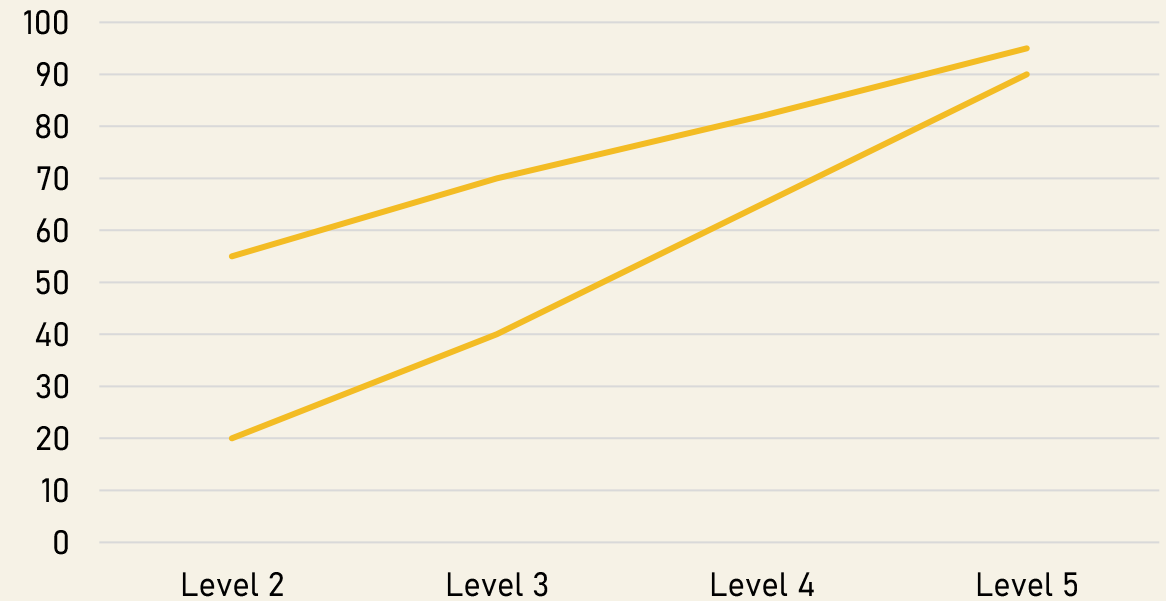


THE TRADEOFF FPN FIXES

Backbone alone forces a tradeoff



FPN SHIFTS THE WHOLE PYRAMID UPWARD IN USEFULNESS



Left: backbone features trade spatial detail for semantics as depth increases. Right: FPN preserves multi-scale resolution while lifting semantic richness at every level, making the full pyramid more useful for detection.



CASE STUDY: AUTONOMOUS DRIVING



A self-driving system must detect both large nearby vehicles and tiny distant hazards in the same frame. Using the backbone alone makes this difficult: deep features carry strong semantics but become too coarse for small faraway objects, while shallow features preserve detail but lack semantic strength. FPN solves this by combining levels so fine-resolution, semantically enriched maps handle small distant objects and coarser maps handle larger nearby objects more reliably.

In driving, multi-scale awareness is not a convenience—it directly affects safety.



FAILURE CASES AND LIMITATIONS

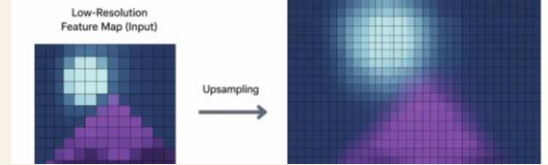


Static grid assumptions

FPN features still live on discrete grids, so irregular or rotated objects may not align well with the sampled locations.

As a result, localization quality can drop when the object geometry conflicts with standard feature-grid assumptions.

FPN improves scale handling, but it does not remove every representation bottleneck.



Upsampling can blur detail

Top-down fusion often enlarges coarse features, which can smooth away sharp edges and fine boundaries.

This helps scale coverage, but it can reduce spatial precision for tasks that depend on exact localization.



Extreme aspect ratios

Very long thin structures are difficult to capture compactly when pyramid levels are tuned for more balanced object shapes.

They may fragment across levels or lose continuity, making representation less stable than for compact objects.



Extra complexity and memory

The added lateral and top-down paths improve multi-scale reasoning, but they also increase feature movement through the network.

That means more compute, more memory pressure, and a heavier deployment footprint than a plain backbone.



WHEN NOT TO USE FPN

- **Is multi-scale detection the main challenge?**

Single-scale sizes: use a simpler detector or plain backbone; FPN may add unnecessary cost.

ALTERNATIVE: WHEN TO MOVE BEYOND FPN

- **PANet**

Adds a second bottom-up path to shorten the flow of low-level localization info to the top layers. Best for precise localization.

- **BIFPN**

Uses weighted bi-directional connections and repetitive stacking for state-of-the-art efficiency. Best for mobile and real time.

- **GraphFPN**

Uses superpixel hierarchies and GNNs to handle non-grid interactions and overlapping objects. Best for irregular clutter.



THANK YOU